AI Tools for Economics Research

Session 1 — Foundations
Monday 25 May 2026
10:30–12:00 · 14:30–16:00
Banco de Portugal
João B. Sousa · Nova SBE

ABOUT THIS COURSE

Working with AI agents on research tasks.

Four sessions, three hours each. Live demos.

Bring your laptop.

THE FOUR SESSIONS

What we'll cover, and when.

Mon 25 May	Foundations — agents and plumbing.
Fri 29 May	Worked examples — research tasks, live.
Mon 22 June	Failure modes and verification.
Tue 23 June	Customising — skills, hooks, subagents, MCP.

Each session: 10:30–12:00 and 14:30–16:00.

TODAY

Before any of that — what is an AI agent?

Not a chat box. Not a smarter autocomplete. A program that lets a language model read your files, run commands, and edit code — under your permission.

01

The landscape.

MANY PRODUCTS, SIMILAR MARKETING

The names we keep hearing.

Claude Code

Codex

Gemini

Copilot

Cursor

THREE LAYERS

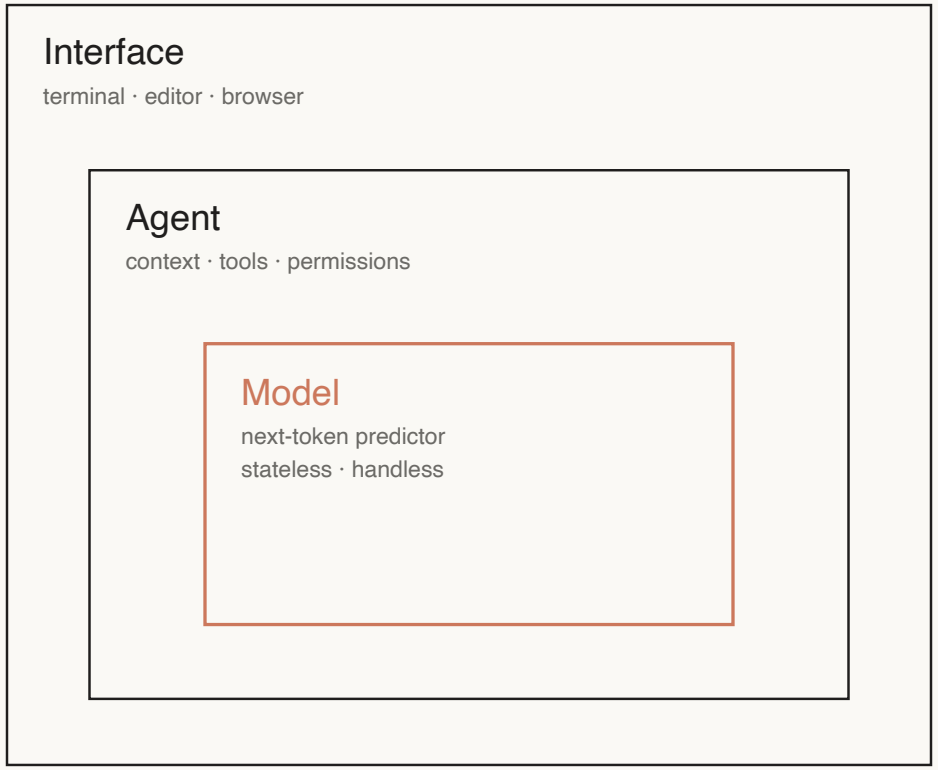
Model. Agent. Interface.

Model the LLM. No memory between turns, no access to your files.

Agent holds context, calls tools, asks permission.

Interface where you type. Browser, editor, terminal.

Same model + different agent = different product.

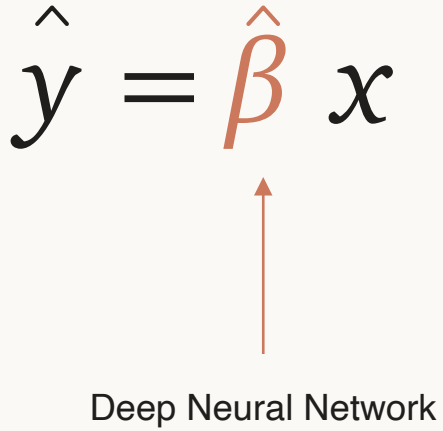


THE PIECE IN THE MIDDLE

What is an LLM?

A probability model over text. You hand it some text; it returns the most likely continuation, one piece at a time.

Same shape as a fitted regression — except the estimator is a deep neural network with billions of parameters.



The agent layer is what makes this useful — next slide.

IF IT'S STATELESS...

...how does it remember anything?

It doesn't. Each turn, the harness — the program wrapping the model — hands it a fresh text bundle: instructions, tools, `CLAUDE.md`, every prior message and tool result, plus your new input. That bundle is the context.

The harness also owns the tool calls, the file permissions, and the project instructions.

Bounded by a context window (200K–1M tokens). Re-sent every turn — drives both cost and rate limits.

WHAT IT LOOKS LIKE

A real session.

```
$ claude

> write the .py code to compute year-over-year
  inflation, save as data/processed/inflation.csv
  plot it to a png file. the data format is:

● Read(data/raw/cpi.csv) — 952 lines
● Write(code/scripts/compute_inflation.py) — 23 lines
● Bash(python code/scripts/compute_inflation.py)
  ⎿ Wrote data/processed/inflation.csv (953 rows)

Done.
```


What a chat box cannot do: open the file, run the code, report back.

MAPPING THE TOOLS

Where each one runs, what it can touch.

TOOL	RUNS IN	CAN TOUCH
Claude / Claude Code	App / Terminal	Uploaded files / Whole filesystem
Codex / Codex CLI	App / Terminal	Uploaded files / Whole filesystem
Gemini / Gemini CLI	App / Terminal	Uploaded files / Whole filesystem
Copilot	Editor	Open files, workspace
Cursor	Editor	Workspace

Pick by where you work, not by which model is "best" this month — most can swap models.

PRIVACY AND DATA HANDLING

What leaves your machine.

The model runs on someone else's servers — Anthropic, OpenAI, Google. No comparable local option.

What the agent reads ships upstream — files, command output, truncated excerpts.

Editor extensions send more than your prompt — Copilot, Cursor ship neighboring files and recent edits as context.

THE LEVER

The agent layer is configurable.

Without retraining the model: `CLAUDE.md`, hooks, skills, MCP servers, permissions.

Sessions 2–4 are about pulling these levers for research workflows.

PAUSE

Questions so far?

02

Chat vs. command-line (CLI) agent.

HOW MOST PEOPLE MEET AI

A box in a browser.

You paste text or upload files. You get text back. You copy the parts you want into a note.

This is fine for many things. It is not what the rest of this course is about.

LIVE DEMO

Chat vs. Claude Code, same prompt.

```
Compute year-over-year inflation from data/raw/cpi.csv.
Save the clean series to data/processed/inflation.csv
and a plot to slides/assets/inflation.png.
```

```
Compute year-over-year inflation from data/raw/cpi.csv.
Save the clean series to data/processed/inflation.csv
and a plot to slides/assets/inflation.png.
```

CHAT	CLI AGENT
Open CSV, paste header.	One prompt with file paths.
Get code, save, run.	Reads, writes, runs.
`ParserError`. Paste back. Rerun.	Same `ParserError`. Self-fixes.
You are the loop.	You review the loop.

WHAT CHANGED

The model can act on the world.

Read files from disk — no copy-paste, no upload limits.

Run commands — `ls`, `grep`, `pytest`, `make`.

Edit code — changes appear as diffs you review.

Three consequences: cost, control, reproducibility.

CONSEQUENCE 1

Cost.

Browser chat: each message ships the whole context again.

CLI agent in your repo: large stable parts of the input — instructions, files, prior turns — are cached at a fraction of the input cost on subsequent turns.

Detail in Session 3. For now: you don't hit your usage cap as fast.

CONSEQUENCE 2

Control.

You see every file the agent reads.

You see every command before it runs (default permission mode).

Every edit is a diff. `git revert` undoes any of it.

CONSEQUENCE 3

Reproducibility.

Edits land in `git`. Commands land in shell history. The full session log can be saved to a file (Session 4 — hooks).

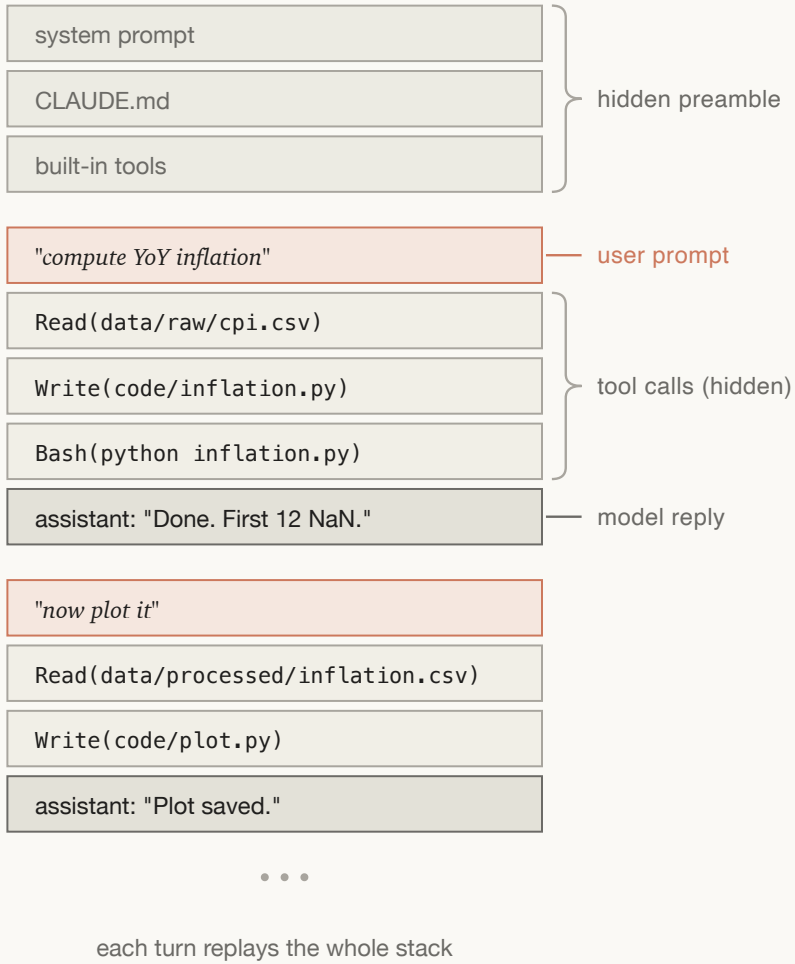
Anyone with the repo can re-run, audit, or take over.

BACK TO THE BUNDLE

The context window is a budget.

Each turn, the harness ships the full conversation back to the model — your prompts, the model's replies, *and every tool result*. The 952-line CSV read earlier stays in the bundle on every subsequent turn.

Bounded at 200K tokens (1M with extended context). Quality degrades well before the limit.



each turn replays the whole stack

END OF MORNING

Where we are.

1. Agent = harness around a stateless LLM. Same model + different harness = different product.
2. Chat → CLI means the model can read, run, edit — and you review the diff.
3. Three consequences: cost (caching), control (diffs and permissions), reproducibility (git and logs).
4. Context is finite — every turn replays the whole bundle.

BREAK

14:30

Back after lunch.

03 · AFTERNOON

The infrastructure underneath.

WHERE THE REST OF THE COURSE LIVES

The agent is useless without filesystem, shell, git, an editor, and permissions.

Five minutes on each. Minimum viable mental model. We'll open a terminal and look at each in turn.

FILESYSTEM

Everything is a file in a folder.

This course repo, for example:

```
bank_of_portugal_ai_2026/
├── README.md, CLAUDE.md
├── slides/              session{1..4}.{md,pdf}, assets/
├── code/
├── data/raw/            cpi.csv  ← from FRED
├── data/processed/
└── docs/                syllabus.pdf
```

The agent reads and writes files. That is the substrate. Nothing else.

SHELL

A program that runs commands.

`ls`	what's here
`cd`	go there
`cat`	read this file
`grep`	find this string
`python clean.py`	run this script

The agent's most-used tool. Whatever you can type, it can type.

GIT

Snapshots of your project over time.

`git status` what's changed since the last snapshot

`git diff` show me the changes, line by line

`git log` what snapshots exist

`git revert` undo a snapshot

Without git, you cannot safely let an agent edit your files.

EDITOR

Where you read the diff before approving.

VS Code is fine. Vim, Emacs, anything you already use is fine. The agent runs in the terminal next to it.

The editor is for you, not the agent.

PERMISSIONS

Trust nothing by default.

Default mode: agent asks before every command and every edit.

You pre-approve safe patterns — `git status`, `git diff`, file reads inside the repo.

Anything destructive — file writes, `rm`, `git push` — still asks.

You pick the working directory. Don't `cd` into folders with supervisory data, credentials, or PII.

04

Setup.

WHAT YOU NEED

Four things on your laptop.

Claude Code installed and authenticated. (Or Gemini CLI, or Codex CLI.)

git installed, with a test repo to play in

GitHub account for cloning and hosting repos

Optional Copilot in VS Code

INSTALL

Get Claude Code running.

```
# macOS / Linux / Windows Subsystem for Linux (WSL)
curl -fsSL https://claude.ai/install.sh | bash

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

claude --version
claude   # first run opens browser for sign-in
```

Paid Anthropic plan required — Pro, Max, Team, or Enterprise. Free plan does not include Claude Code.

docs.anthropic.com/claude-code

SMOKE TEST

Does it work?

```
cd ~/bank_of_portugal_ai_2026
claude
> What's in this repo?
```

The agent should describe the `slides`, `code`, `data`, and `docs` folders — actually read, not guess. If it does, you are running.

Three ideas to carry into Session 2.

1. An agent is a harness around an LLM. The model is stateless; the harness is what you configure.
2. The shift from chat to CLI is the model being able to read, run, and edit — with diffs and git as safety net.
3. The agent is only as good as the filesystem, shell, and git underneath it. On Friday we use all three on real research tasks.

See you on Friday.

joao.sousa@novasbe.pt